Mohammed VI Polytechnic University
Rabat Campus
Mock Midterm Examination

Compilers: Lexical Analysis, Grammars and Top-down Parsing

Name:		**Student ID:**	
Date:		**Duration:**	120 minutes
Total points:	100	**Problems:**	5

Instructions

- Answer all 5 problems. Points are shown next to each part; the exam totals 100 points. Write your answers on separate answer sheets, numbering them as in the exam.
- Show your work. Partial credit is given for correct intermediate results (FIRST/FOLLOW sets, transformed grammars, stack traces, and so on).
- **Multiple-choice questions:** choose exactly one option **and explain why you chose it**. Credit is given for both the option and the explanation; a correct option without an explanation earns only partial credit.
- Notation: `$` (or `EOF`) is the end-of-input marker and ε is the empty string. Write stacks with the top at the right-hand end. Number the productions of a grammar top to bottom when you build a parse table.
- Materials and permitted aids: as announced by your instructor.

Point distribution

Problem	Topic	Points	Score
1	Short questions and multiple choice	30	
2	Lexical analysis: scanner rules, DFA, comments	22	
3	LL(1) parsing: grammar conditioning, FIRST/FOLLOW, table	19	
4	LL(1) parsing with lookahead: code, parse tree and stack	16	
5	LL(1) table and parsing actions	13	
Total		**100**	

Problem 1: Short Questions and Multiple Choice (30 points)

Answer each question briefly. For every multiple-choice question choose exactly one option **and explain why you chose it**.

1.1 (3 pts) Which statement correctly describes a context-free grammar G = (N, T, S, P)?

- **(A)** Every production has the form `A → w` with a single nonterminal A on the left and w a (possibly empty) string of terminals and nonterminals.
- **(B)** Every production has the form `α → β` where α and β are arbitrary strings and |α| ≤ |β|.
- **(C)** Every production has the form `A → aB` or `A → a`, with a a terminal.
- **(D)** The grammar is context-free exactly when it is unambiguous.

Explanation (required):

1.2 (4 pts) Let L = { `a^n b^(2n)` : n ≥ 0 } (for example ε, `abb`, `aabbbb`). (a) Argue that L is not regular (3 pts). (b) Give a context-free grammar for L (1 pt).

1.3 (4 pts) Consider the grammar with start symbol `S` whose terminals are `a b c d e f`:

```
S → a A B c
A → A b d | e
B → f
```

For the sentence `a e b d b d f c` give (a) a leftmost derivation (1 pt), (b) a rightmost derivation (1 pt), and (c) the parse tree (2 pts).

1.4 (7 pts) Consider the following grammar for subtraction expressions, with start symbol `E` and terminals `NUM`, `-`, `(` and `)`:

```
E → E - E | ( E ) | NUM
```

(a) Show that the grammar is ambiguous by giving two different parse trees for `NUM - NUM - NUM` (1 pt). (b) If this input stands for `10 - 4 - 3`, what value does each tree give? Which tree should a compiler choose, and which associativity does it correspond to? (2 pts) (c) Give an unambiguous grammar for the same language in which `-` is left-associative (3 pts). (d) Your grammar from (c) is left-recursive. Rewrite it as an LL(1) grammar and say in one sentence why the left-recursive version cannot be used by a predictive parser (1 pt).

1.5 (4 pts) Consider the grammar with start symbol `S`:

```
S → A x | B x
A → a | ε
B → b | ε
```

(a) Compute FIRST and FOLLOW for `S`, `A` and `B` (2 pts). (b) In which cell of the LL(1) table is there a conflict? Choose one option and explain why (2 pts).

- **(A)** `M[S, a]`
- **(B)** `M[S, b]`
- **(C)** `M[S, x]`
- **(D)** None; the grammar is LL(1).

Explanation (required):

1.6 (4 pts) Eliminate the ε-production from the grammar `S → ( S ) S | ε` without changing its language. (a) Which nonterminals are nullable (1 pt)? (b) Give the resulting grammar; if ε is in the language, keep it through a new start symbol (3 pts).

1.7 (4 pts) (a) Which statement about NFAs and DFAs is correct? Choose one option and explain why (2 pts).

- **(A)** NFAs recognize more languages than DFAs, which is why scanner generators prefer them.
- **(B)** A DFA may have several transitions on the same symbol out of one state; that is the only difference from an NFA.
- **(C)** The subset construction always produces a DFA with exactly as many states as the NFA.
- **(D)** Every NFA can be converted into a DFA that accepts the same language, but the DFA may need exponentially more states than the NFA.

Explanation (required):

(b) (2 pts) Write a regular expression for numeric literals that consist of an integer part, an optional fraction and an optional exponent. Examples that must match: `42`, `3.14`, `6.02E23`, `1.5e-3`; examples that must not match: `.5`, `3.`, `1e`. Use `[0-9]` for a digit.

Problem 2: Lexical Analysis (22 points)

A scanner generator (such as `flex`) is given the rules below, listed in this order. When several rules match, the scanner returns the rule with the **longest** match; if two rules match a lexeme of the same length, the rule listed **first** wins. A character that cannot start any token is reported as a lexical error and skipped.

```
if                  IF
[a-z][a-z0-9]*      ID
[0-9]+              INT
[0-9]+\.[0-9]+      REAL
<                   LT
<=                  LE
<>                  NE
[ \t]+              (skip)
```

2.1 (5 pts) Give the sequence of tokens (token name and lexeme) that the scanner returns for the input below, leaving out skipped whitespace. State where the lexical error occurs.

```
if iff<=3.14<>x12 if3 7.
```

2.2 (7 pts) Draw the single DFA that the scanner generator builds for all eight rules together. Use the character classes `i`, `f`, `L` (any other lowercase letter), `D` (a digit), `.`, `<`, `=`, `>`, and `ws` (space or tab). Label every accepting state with the token it returns, and state (in one or two sentences) how the DFA handles the lexeme `if`, which matches both `if` and the identifier rule, and how the scanner implements the longest-match rule.

2.3 (6 pts) A comment starts with `/*` and ends at the **first** `*/` that follows. Use the reduced alphabet { `/`, `*`, `x` } where `x` stands for any other character.

(a) Write a regular expression for comments (2 pts). (b) Draw a DFA that accepts exactly the comments (3 pts). (c) Can a DFA accept **nested** comments, in which `/* a /* b */ c */` is one comment? Explain briefly (1 pt).

2.4 (4 pts) The rule `[a-z][a-z0-9]*  ID` is moved **above** the rule `if  IF`; nothing else changes. What does the scanner now return for the inputs `if` and `iff`? Choose one option and explain why.

(A) `ID` for `if` and `ID` for `iff`.

(B) `IF` for `if` and `ID` for `iff` (the longest-match rule makes the order irrelevant).

(C) `ID` for `if` and `IF` for `iff`.

(D) A lexical error for both inputs, because two rules now match.

Explanation (required):

Problem 3: LL(1) Parsing (19 points)

Consider the following grammar for nested blocks of simple statements:

```
Block  → begin Stmts end
Stmts  → Stmts Stmt
Stmts  → ε
Stmt   → Block
Stmt   → Simple
Simple → id ;
```

where `begin`, `end`, `id` and `;` are terminal symbols, all other symbols are nonterminal symbols, and `Block` is the start symbol.

3.1 (8 pts)

(a) (4 pts) Which of the following is the best explanation of why the grammar as given is **not** LL(1)? Choose one option and explain why you chose it (and why you rejected the others).

(A) The grammar is ambiguous: some string has two different parse trees.

(B) The two alternatives of `Stmt` both begin with the same terminal, so their FIRST sets overlap.

(C) The nonterminal `Stmts` is left-recursive, so with one token of lookahead the parser cannot choose between `Stmts → Stmts Stmt` and `Stmts → ε`.

(D) The language is not context-free, so no LL(1) grammar for it can exist.

Explanation (required):

(b) (4 pts) Transform the grammar into an equivalent form that is LL(1). Do **not** introduce any new nonterminals.

3.2 (5 pts) Compute the FIRST and FOLLOW sets for all nonterminals of your LL(1) grammar from 3.1(b).

3.3 (6 pts) Construct the LL(1) parse table for your grammar. Number the productions of your grammar in the order you wrote them (top to bottom) and use these numbers as the table entries.

Problem 4: LL(1) Parsing with Lookahead: Code, Parse Tree and Stack (16 points)

The following recursive-descent parser has one token of lookahead (`lookahead` holds the current token, `nextToken()` reads the next one, and `EOF` is returned at the end of the input). The tokens are `ID`, `ASSIGN (:=)`, `SEMI (;)`, `PRINT`, `PLUS (+)`, `NUM`, `LPAREN`, `RPAREN` and `EOF`.

```
void S() {
  if      (lookahead == ID)    { match(ID); match(ASSIGN); E(); match(SEMI); S(); }
  else if (lookahead == PRINT) { match(PRINT); E(); match(SEMI); S(); }
  else if (lookahead == EOF)   { /* nothing */ }
  else error();
}
void E()  { T(); Eprime(); }
void Eprime() {
  if (lookahead == PLUS) { match(PLUS); T(); Eprime(); }
  /* otherwise: return without doing anything */
}
void T() {
  if      (lookahead == ID)     match(ID);
  else if (lookahead == NUM)    match(NUM);
  else if (lookahead == LPAREN) { match(LPAREN); E(); match(RPAREN); }
  else error();
}
void match(Token t) {
  if (lookahead == t) lookahead = nextToken(); else error();
}
```

4.1 (3 pts) Write down the grammar this code implements (use `E'` for `Eprime` and ε for an empty alternative), and for **each production** give the set of lookahead tokens that selects it (for an ε-alternative, give the tokens that can legitimately follow the nonterminal).

4.2 (4 pts) The input is `id := ( num + id ) ; print num ;`. (a) Draw its parse tree (3 pts). (b) List, in order, the procedure calls (`S()`, `E()`, `match(...)`, and so on) made while the **first statement** is parsed (1 pt).

4.3 (4 pts) Parse the input `print ( id ) + num ;` with the non-recursive (stack-based) predictive parser that corresponds to this code. Use your answers to 4.1 as the parsing table. For each step give the stack (top at the right-hand end, starting from `$ S`), the remaining input, and the action taken.

4.4 (2 pts) For each of the erroneous inputs `id := num num ;` and `id := + ;`, state (i) in which procedure and in which call the **recursive** parser detects the error, and (ii) in which table cell the **stack-based** parser detects it. If the two parsers detect the error at different places for the same input, explain why.

4.5 (3 pts) Suppose the rule for `E` were `E → E + T | T` and a student wrote `void E() { E(); match(PLUS); T(); }` for its first alternative. What happens? Choose one option and explain why.

- **(A)** The parser accepts the same language as before.
- **(B)** The parser calls `E()` again without consuming any token, so it recurses forever (stack overflow).
- **(C)** The parser rejects every input because `match(PLUS)` always fails.
- **(D)** The parser reports an error on the first token, because `E()` has no `error()` branch.

Explanation (required):

Problem 5: LL(1) Table and Parsing Actions (13 points)

Given the grammar G with start symbol `S`, nonterminals `S, P, Q, R` and terminals `( ) x y z w`:

```
S → P Q R
P → ( S ) | x S y | z
Q → x Q | ε
R → w R | ε
```

(a) (8 pts) Construct a table-based LL(1) predictive parser for G: compute the FIRST and FOLLOW sets and fill in the LL(1) table. Does the grammar need to be modified first? Justify your answer.

(b) (5 pts) Give the parsing actions (stack, remaining input, action) for the input string `( z w ) x w`. Write the stack with its top at the right-hand end.

— End of exam —